

수성록 (Tower and Dragons) — 최종 결과 보고 발표 대본

총 분량: 약 23분 00초 (슬라이드 32장) **기준 자료:** Docs/최종발표/신버전_수성록_최종결과보고_디자인 적용_2.pptx **대조 이력:** 2026-09-02, 30장 판본(수성록_최종결과보고_디자인적용_1.pptx)의 슬라이드 텍스트와 1:1 대조해 슬라이드 4·5·7·12·13·17·18·20·24·25의 화면 지시와 수치를 PPT 실물에 맞춰 수정했습니다. **발표자:** 나상욱 (표지 하단에는 팀원 4인 공동 표기) **읽기 속도:** 분당 약 300자 기준. 실제 리허설에서 ±10% 오차를 감안할 것 **표기:** [화면] = 슬라이드 조작 지시 / [강조] = 반드시 짚고 넘어갈 지점

시간 배분 요약

구간	슬라이드	시간	누적
도입	1~2	0:50	0:50
01 프로젝트 개요	3~9	3:55	4:45
02 팀 구성 및 역할	10~13	1:45	6:30
03 수행 절차 및 방법	14	1:15	7:45
04 핵심 기술 구현	15~26	11:50	19:35
04 완성도 · 시연	27~29	1:20	20:55
05 자체 평가	30~32	2:05	23:00

총 **32장** · 약 23분.

기준 파일이 바뀌었습니다. 이전 29·30장 판본에서 두 장이 늘었습니다. **13번**(협업 구조 아이콘 버전)과 **22번**(자체 제작 에디터 도구 — 실행 화면)이 신규이고, 그만큼 뒤쪽 번호가 전부 밀렸습니다. **옛 번호로 외우지 마세요.**

슬라이드에 표·타임라인·다이어그램이 함께 올라가 있으므로, 대본은 **화면을 짚어가며 읽는 부분**과 **화면에 없는 내용을 구두로 보태는 부분**이 섞여 있습니다. > 표시가 붙은 안내문을 먼저 읽고 그 슬라이드를 준비하세요.

04장 순서: 결합도 → 컴포지션 → SO 조합 → 이벤트 기반 아키텍처 → 에디터 툴 파이프라인 → **자체 도구 카드** → **자체 도구 실행 화면(신규)** → Claude 스킴 → 최적화 → 트러블슈팅 → 디자인 패턴.

20분을 맞춰야 하면 부록 B의 압축 지침을 적용해 약 3분을 덜어내세요.

시연 영상을 발표 중에 재생하는 경우 — 29번 슬라이드에서 8분이 추가되므로 총 31분이 됩니다. 20분을 지켜야 한다면 부록 B를 참고하세요.

도입 (0:50)

▶ 슬라이드 1 — 표지

[화면] 표지 슬라이드

안녕하십니까. 이번 디벨로켓 기업 협약 프로젝트 수성록**3팀**의 발표를 맡은 **나상욱**입니다.

저희 팀이 수성록 미션 가이드를 바탕으로 개발한 「Tower and Dragons」의 최종 결과 보고를 시작하도록 하겠습니다.

오늘 발표에서는 저희가 무엇을 기획했고, 그중 무엇을 실제로 구현했으며, 기획대로 되지 않은 부분은 왜 그렇게 바뀌었는지를 중심으로 말씀드리겠습니다.

▶ 슬라이드 2 — 목차

[화면] 목차

발표는 보이시는 다섯개의 목차로 구성했습니다.

01 프로젝트 개요 — 주제와 차별화, 구현 규모, 기획 변경 사항 **02 팀 구성 및 역할** — 역할과 담당 영역, 협업 구조 **03 수행 절차 및 방법** — 총 9주 개발 일정과 실행 내역 **04 프로젝트 수행 경과** — 아키텍처, 설계와 패턴, 최적화, 개발 도구, 시연 **05 자체 평가 의견** — 완성도 평가와 개선점, 느낀 점

01 · 프로젝트 개요

▶ 슬라이드 3 — 프로젝트 개요 간지

▶ 슬라이드 4 — 프로젝트 개요

[화면] 슬라이드 4 - 컨셉 문장 + 장르 부제 + 하단 메타 카드 4개(장르 / 플랫폼 / 개발 / 규모)

화면은 컨셉 문장 한 줄 + 장르 부제, 그리고 하단 메타 카드 네 칸입니다. 카드는 한 번에 훑고 넘기고, 해석은 구두로 채웁니다. 30초 안에 끊고 5번으로 넘어가세요.

먼저 프로젝트 개요입니다. [화면 상단 컨셉 문장을 짚으며] 저희 게임의 한 줄 컨셉은 "낮의 선택이 밤의 생존을 결정한다"입니다.

[강조] 이 한 문장이 타워 앤 드래곤즈의 전체적인 주제를 관통합니다.

[하단 카드 네 칸을 왼쪽부터 한 번에 훑으며] 장르는 타워디펜스에 도시건설을 결합한 싱글플레이 전략 게임, 플랫폼은 Windows PC, 개발은 Unity와 C# 기반의 데이터 주도 설계, 규모는 9주 · 4인입니다.

저희는 수성록의 미션가이드를 "낮에 한정된 자원을 전략적으로 투자해 밤을 대비한다"로 해석했습니다. 따라서 타워 앤 드래곤즈의 플레이어는 낮에는 시간 제한이 없이 얼마든지 고민하고 행동할 수 있습니다. 대신 밤에는 개입할 여지가 거의 없으며 낮에 내린 판단이 그대로 결과로 돌아오게 됩니다.

▶ 슬라이드 5 — 차별화 포인트

[화면] 슬라이드 5 - 상단 게임 화면 3장 + 하단 카드 3개 (용의 운용 / 전략적 확장 / 인구의 배분)

화면에 세 요소가 이미지와 카드로 나란히 올라가 있습니다. 카드의 글자를 읽지 말고, 위쪽 이미지를 왼쪽부터 하나씩 짚으면서 아래 내용을 말하세요.

방금 말씀드린 컨셉은 그렇게 특별한 개념은 아니기에 저희 팀은 타워디펜스라는 장르에 더해서 타워앤드래곤즈만의 차별성을 아래의 세가지요소로 주었습니다.

[왼쪽 이미지를 짚으며] **첫째, 용의 운용입니다.** 기본적인 타워디펜스 장르에 어미용과 새끼용을 운용하면서 RPG의 성장 요소와 플레이어만의 빌드를 만들 수 있는 요소인 용 스킬 트리 시스템을 적용했습니다. 어미용은 낮에 변경할 수 있으며 게임의 필드 전역에 영향을 줍니다. 새끼용은 필드에 타워처럼 설치되어 플레이어가 전략적 요소로 운용할 수 있습니다. 또한 도시 건설 장르에서 참조를한 연구 시스템 또한 플레이어의 전략 요소로 용의 스킬 트리와 함께 작용합니다.

[가운데 이미지로 옮기며] **둘째, 전략적 확장입니다.** 플레이어는 땅을 점령해야만 게임의 핵심적인 자원인 인구와 게임의 후반부 진행에 필수인 특화 자원과 신규 타워를 얻을 수 있습니다. 하지만 점령에는 인구와 자원이 필요하며, 여러날이 소요됩니다. 거기에 더해 등장하는 적들의 종류와 수가 증가하게 되어있어 성장과 위험이 같은 축에 묶여있습니다. 타워 디펜스에 이러한 전략적 차별화 요소를 주었습니다. [카드 아래 꼬리말을 가리키며] 그래서 점령은 **인구의 유일한 수급처**이기도 합니다.

[오른쪽 이미지로 옮기며] **셋째, 인구의 배분입니다.** 저희 게임에서 인구는 생산, 방어, 점령, 연구 네 곳이 **하나의 풀을 공유**합니다. 미션 가이드에서 나온 방어 혹은 운영에 인구를 할당하는 컨셉을 확장하여 생산, 영토 확장, 방어, 성장 모든 요소에 인구가 필요하도록 설계하였습니다. 따라서 플레이어는 매일 인구를 어떻게 효율적으로 활용할것인지 고민하게 됩니다. [카드 아래 꼬리말을 가리키며] 화면에 적힌 대로 인구가 **전략적 선택의 핵심 자원**입니다.

[강조] 이 세 가지는 각각 따로 노는 기능이 아니라 서로 물려 있습니다. 매일 매일 플레이어는 인구와 자원을 관리하며 방어해야하고, 새롭게 얻은 자원과 용들로 성장하여 클리어까지 향하게 됩니다.

저희 팀은 타워 디펜스라는 장르를 중심으로 이 세개의 요소들이 유기적으로 연결되어 플레이어에게 익숙하지만 새로운 게임플레이를 경험할 수 있도록 게임을 개발 했습니다.

▶ 슬라이드 6 — 핵심 지표 · 구현 규모

[화면] 슬라이드 6 - 상단 4개 지표 카드 + 하단 구현 규모 표

슬라이드는 위쪽 카드 네 개와 아래쪽 구현 규모 표 6행으로 되어 있습니다. 카드 네 개만 읽고, 표는 훑으면서 두 세 항목만 짚으세요. 전부 읽으면 시간이 넘어갑니다.

타워 앤드래곤즈는 전체적으로 다음과 같은 다양한 구성 요소들로 이루어져 있습니다. 플레이어가 다양한 전략과 빌드를 구성하고 경험할 수 있도록 프로젝트 기간내에 최대한 많은 콘텐츠를 담으려고 노력했습니다. 타워 디펜스라는 장르를 중심으로 타워앤드래곤즈의 차별화 포인트들이 플레이어에게 확실한 재미를 주기 위해 다양한 타워와 몬스터, 용, 맵의 지형과 자원들을 구현했습니다.

▶ 슬라이드 7 — 구현 규모 · 타워와 몬스터 라인업 (0:20)

[화면] 슬라이드 7 - 좌: 타워·건물 아이콘 그리드 / 우: 몬스터 34종 그리드

읽을 것이 없는 장입니다. 숫자는 앞 장에서 이미 말했으니 여기서는 화면을 한 번 훑게 하고 20초 안에 넘기세요. 개별 이름을 부르지 마세요.

[왼쪽 그리드를 손으로 한 번 훑으며] 왼쪽은 전투 타워 13종과 건물입니다. 기본 타워 세 종에 대공 한 종, 속성 다섯 종, 그리고 은신·영혼·가시·강화의 특수 네 종입니다. 속성 타워와 특수 타워가 뒤에 나올 데이터 조합 이야기의 소재가 됩니다.

[오른쪽 그리드로 옮기며] 오른쪽은 몬스터 34종입니다. 탱커와 원거리, 자폭, 마비, 힐러, 보호형, 그리고 속성 면역까지 **역할이 겹치지 않도록** 나눠 만들었습니다.

▶ 슬라이드 8 — 구현 규모 · 주요 콘텐츠 화면 (0:20)

[화면] 슬라이드 8 - 용 스킬트리 / 연구 트리 / 점령 플레이 화면

마찬가지로 **훑고 넘기는 장**입니다. 이 세 화면이 뒤에서 계속 인용되므로 "**이게 그 화면이다**"라고 눈에 한 번 넣어주는 것이 목적입니다.

[세 화면을 차례로 짚으며] 용 스킬트리와 연구 트리, 그리고 점령 화면입니다. 왼쪽 두 개가 플레이어의 성장 축이고, 오른쪽이 영토를 넓히는 축입니다.

[강조] 이 세 화면은 뒤에서 다시 나옵니다. 스킬트리 70노드와 연구 38노드는 **사람이 손으로 만든 게 아니라 도구로 찍어낸 것**인데, 그 이야기는 21번과 22번 슬라이드에서 말씀드리겠습니다.

▶ 슬라이드 9 — 기획 변경 사항 및 사유

[화면] 슬라이드 9 - 변경 사항 표 5행 + 하단 "변경 원칙" 띠

표의 ****첫 행**이 "자원 종류 감소"입니다. 아래 순서가 표 순서와 같으니 그대로 읽으세요.

표는 **초기 기획 / 변경된 구현 / 변경 사유** 세 칸입니다. 표의 위에서 아래 순서 그대로 말하세요.

이번에는 초기 기획대로 하지 않은 부분을 말씀드리겠습니다. 다섯 가지입니다.

첫째, 자원 종류를 기본 4종에서 3종으로 줄였습니다. 광물을 뺐습니다. 자원 HUD를 봤을 때 한눈에 안 들어오고, 초반 관리 피로만 늘리고 있었습니다.

둘째, 주둔지 시스템을 제외했습니다. 원래는 점령하면 주둔지가 지어지고 그걸 강화하는 구조였는데, 건물이 하나 더 늘어나는 만큼 관리 피로도가 커서 **점령 보상을 즉시 지급**하는 방식으로 바꿨습니다. 불필요한 마이크로매니지먼트를 제거한 겁니다.

셋째, 타워 파괴를 비활성화 후 인구 총원 부활로 바꿨습니다. 밤 동안에 인구가 없는 타워를 앞에 설치해서 무한으로 막을 수 있는 플레이 방식이 빌드 테스트 중에 나오게 되었고 이로 인해 게임 진행이 불가한 상황이 발생했습니다.

넷째, 적 타겟팅에 새끼용을 포함시켰습니다. 원래 적은 성과 타워만 공격했는데, 그러면 새끼용을 아무 데나 놔도 손해가 없습니다. 새끼용 배치에 리스크와 리턴을 부여한 변경입니다.

다섯째, 점령 페널티를 적 스탯 배율에서 추가 스폰량 중심으로 전환했습니다. 배율이 곱연산으로 중첩되면서 난이도가 비선형으로 급증하는 문제를 해소했습니다.

[강조] 하단의 "변경 원칙" 한 줄을 가리키며] 위 변경 사항들의 공통점이라고 하면 최대한 게임의 코어에만 집중하려고 했다는 겁니다. 저희가 이 게임에서 추구하는 핵심적인 재미요소들을 방해하거나 재미를 주는데에 큰 영향이 없는 요소들은 과감하게 제외했습니다.

02 · 팀 구성 및 역할 (1:45)

▶ 슬라이드 10 — 팀 구성 및 역할 간지

▶ 슬라이드 11 — 팀 구성 및 역할 (0:50)

[화면] 슬라이드 11

슬라이드에는 이름 · 역할 · 담당 영역 세 칸만 있습니다. **세부 담당은 구두로** 채우세요.

저희 팀은 팀장의 중앙 통제와 각 팀원들의 치밀한 역할분담으로 개발의 효율성을 극대화 시켰습니다.

조강현 팀원은 팀장을 맡으면서 시스템 인프라와 중앙에서 PR 및 코드리뷰를 핵심으로 담당했습니다. 세이브·로드 시스템을 단독으로 구현했고, 설정과 키 리바인딩, 프로젝트 초기 골격과 매니저 구조, 미니맵, 전장의 안개, 툴팁, 도움말 도감, 가이드 퀘스트를 맡았습니다. 씬 배선 실수를 자동으로 잡아내는 점검 도구도 직접 만들었습니다.

김지해 팀원은 UI와 아트 연출을 담당했습니다. 인게임 UI 프리팹과 스프라이트를 제작했고, 성 체력과 게임오버 연출, 점령·자원·연구 창, 용 스킬트리 UI 연출, 주민 캐릭터 이동 연출을 맡았습니다.

이하늘 팀원은 낮 운영과 온보딩을 담당했습니다. 그리드와 건물 배치 시스템, 점령 시스템, 자원 노드, 봉인석 엔딩, 그리고 튜토리얼 전체와 타워·몬스터 이펙트, 전투 효과음을 맡았습니다.

나상욱 팀원은 밤 전투와 밸런스를 담당했습니다. 웨이브, 포탈, 몬스터, 타워 시스템과 인구 시스템, 점령 페널티, 용 시스템, 연구 시스템, 그리고 밸런스 데이터와 시뮬레이션 툴을 맡았습니다.

▶ 슬라이드 12 — 협업 구조 · 수치 (0:25)

[화면] 슬라이드 12 — 70%+ 역할 분담 / 2곳 충돌 최소화 / 83.6% PR 리뷰 단일화

12번과 13번은 같은 내용의 두 판본입니다. 12번이 수치 카드, 13번이 아이콘 버전입니다. 여기서는 **수치 세 개만 던지고 곧바로 13번으로 넘어가** 설명을 붙이세요. (두 장을 다 쓸 이유가 없다면 **한 장을 지우고 시간을 아끼는 편이 낫습니다.**)

이 슬라이드는 **Git 커밋과 PR 110건을 데이터로 분석해서** 확인한 협업 구조입니다.

[강조] 저희 팀은 계획한 일정보다 **항상 조금씩 앞서 진행됐는데**, 저희는 이것을 협업 구조의 결과라고 봅니다.

[세 수치를 왼쪽부터 **짚으며**] 전담 영역 점유율 **70퍼센트 이상**, 네 명의 작업 구역이 겹친 지점은 **단 2곳**, 팀장이 직접 리뷰·머지한 PR 비율 **83.6퍼센트**. 하나씩 말씀드리겠습니다.

▶ 슬라이드 13 — 협업 구조 · 세 축 (0:50)

[화면] 슬라이드 13 — 역할 분담 / 충돌 최소화 / PR 리뷰 단일화 아이콘 3열

앞 장과 수치가 같습니다. **여기서 살을 붙입니다.** 아이콘을 왼쪽부터 짚으세요.

[첫 번째, 역할 분담] **첫째, 역할 분담입니다.** 네 명이 각자 전담 영역을 **70퍼센트 이상 단독으로** 작업했습니다. 세이브·로드 100퍼센트, 튜토리얼은 99퍼센트, 점령 밸런스 데이터는 93퍼센트입니다. **명확한 업무 경계 덕에 중복 작업과 병목이 없었습니다.**

[강조] 참고로 이 점유율은 코드 라인 수가 아니라 해당 영역 파일을 수정한 횟수 기준입니다. Unity의 프리팹이나 씬 파일은 인스펙터 값 하나만 바뀌어도 수천 줄이 갈리기 때문입니다.

[두 번째, 충돌 최소화] 둘째, 충돌 최소화입니다. 작업 단위마다 브랜치를 끊고, 동시에 손대는 구역이 겹치지 않게 미리 나눠 두고 계속 소통했습니다. 그 결과 9주 동안 네 명이 같은 시간에 작업하면서도 **공동으로 손댄 구역은 화면에 적힌 대로 단 2곳**이었고, Git 충돌이 사실상 없었습니다. [여유가 있으면 한 문장] 그렇게 끊어서 운영한 브랜치가 **102개**입니다.

[세 번째, PR 리뷰 단일화] 셋째, PR 리뷰 단일화입니다. 전체 PR 110건 중 92건, **83.6퍼센트**를 팀장이 직접 리뷰 하고 머지했습니다. 나머지는 팀원들이 진행한 UI와 씬, 데이터 변경이었습니다. **코드 품질과 컨벤션을 한 창구에서 일원화**한 겁니다.

03 · 수행 절차 및 방법 (1:15)

▶ 슬라이드 14 — 수행 일정 및 실행 내역 (1:15)

[화면] 슬라이드 14 - 좌우로 교차하는 수직 타임라인 6단계

6단계와 날짜는 **화면에 이미 다 적혀 있습니다. 단계를 하나씩 읽지 마세요.** 강조는 일정표가 아니라 "**이 9주가 굴러간 방식**" 에 두고, 타임라인은 손으로 한 번 훑으면서 **세 구간으로 묶어** 말합니다.

전체 일정은 7월 3일부터 9월 3일까지 **총 9주**이고, 화면의 6단계는 저희가 **주 단위로 끊은 빌드 여섯 번**에 그대로 대응합니다.

[강조] 단계 목록보다, 이 여섯 번이 어떻게 굴러갔는지를 말씀드리겠습니다.

빌드마다 **계획서를 먼저 쓰고, 빌드를 내고, 결과 보고서를 남겼습니다.** 그 보고서에서 확인된 공백이 그대로 다음 주의 작업 우선순위가 됐습니다. 할 일을 회의가 아니라 **문서로 정하는 사이클**을 9주 내내 반복한 겁니다.

[타임라인을 위에서 아래로 한 번에 훑으며] 그 결과가 화면의 흐름입니다. 크게 **3주씩 세 구간**입니다. **첫 3주**는 만들기 전에 컨셉과 아키텍처를 먼저 세운 구간, **다음 3주**는 코어 루프의 재미를 검증한 뒤 용·타워·랜드마크까지 전 기능을 연결한 구간, **마지막 3주**는 밸런스 와 튜토리얼, 연출을 다듬어 완성도를 올린 구간입니다.

마디는 **컨셉 발표 · 중간 보고 · 최종 제출** 세 시점이었고, 각 시점에 무엇이 반드시 돌아가야 하는지를 역산해 주간 빌드를 배치했습니다.

04 · 핵심 기술 구현 (11:50)

▶ 슬라이드 15 — 간지 (0:15)

[화면] 간지 슬라이드

다음은 저희가 이번 프로젝트를 진행하면서 게임에 적용한 결합도 분리와 데이터 조립 등 핵심적인 기술에 관해 설명하도록 하겠습니다. 시행착오를 겪으며 여러가지 디자인 패턴을 차용하고, 성능저하 등의 사고를 미연에 방지하기 위해 다양한 최적화 방식을 도입하는 등 탄탄한 아키텍처를 구성하려고 노력을 했습니다.

▶ 슬라이드 16 — 결합도 분리 · 의존성 역전 (1:00) ★

[화면] 슬라이드 16 - 좌우 비교 다이어그램 ("직접 참조했다면" ↔ "질의 인터페이스로 뒤집었다")

청중이 가져가야 할 것은 사례 개수가 아니라 **화살표 방향이 뒤집혔다**는 사실 하나입니다.

첫번째로 결합도 분리와 의존성 역전입니다. 예를 하나만 들겠습니다.

타워 하나의 공격력에 네 개의 시스템이 동시에 개입합니다. 연구로 강화되고, 어미용 패시브가 붙고, 새끼용 버프가 걸리고, 지형 페널티가 깎습니다.

[화면 왼쪽을 짚으며] 네 시스템이 타워 스탯을 직접 고치게 하면 **타워가 네 시스템을 전부 알아야 합니다**. **if** (연구 해금됐나), **if** (용 버프 있나) 같은 분기가 타워 안에 쌓이고, 버프를 하나 추가할 때마다 타워 코드를 또 고쳐야 합니다.

[화면 오른쪽으로 시선을 옮기며] 그래서 **의존 방향을 뒤집었습니다**. `ITowerStatMultiplierQuery`라는 **질의 인터페이스 하나**를 두고, **버프를 주는 쪽이 그걸 구현하게** 했습니다. 타워는 "내 배율이 얼마나"고 묻기만 하고, 누가 답하는지는 모릅니다.

[강조] 화살표 방향이 뒤집힌 것이 핵심입니다. 타워가 연구와 용을 알던 구조에서 **연구와 용이 타워의 인터페이스를 아는 구조로 바뀌었습니다**. 그래서 새 버프를 붙여도 **타워 코드는 한 줄도 고치지 않습니다** — **개방-폐쇄 원칙, OCP**입니다.

같은 질의 인터페이스를 프로젝트 전체에 **30개** 뒀습니다.

▶ 슬라이드 17 — 컴포지션 기반 데이터 주도 설계 (1:35)

[화면] 슬라이드 17 - 3층위 다이어그램

이 장은 **주장**이고, 바로 다음 18번이 **그 증거**입니다. 두 장을 붙여서 말하세요.

데이터 주도 설계라고 하면 보통 **수치**만 밖으로 빼는 걸 말합니다. 저희는 **행동까지** 뺐습니다. 세 층위로 조립합니다.

① **컴포넌트 조합**. 타워 13종이 전부 같은 `Tower` 클래스입니다. 모든 타워는 상속으로 종류가 나뉘는게 아니라 어떤 컴포넌트를 붙이느냐로 나뉩니다. 특정 특수 타워는 인구 없이 가동돼야 하는데, `TowerPopulation` 대신 `AlwaysOnStaffing`을 붙이면 끝입니다. **타워 코드에는 특수 타워를 위한 분기가 하나도 없습니다**.

② **데이터 조합**. `TowerData`가 `AttackSO`를 참조하고, 그게 `AttackEffectSO`를, 그게 다시 `StatusEffectSO`를 참조합니다. SO가 SO를 참조하는 트리라서, **새 타워는 기존 SO들의 새로운 조합**입니다.

③ **행동 조합**. `SpecialBehaviorRunner`가 `SpecialBehaviorSO` 목록을 실행합니다. 자폭, 보호막 부여, 보호막 재생 같은 특수 행동을 몬스터 클래스를 **상속하지 않고** 데이터로 붙입니다.

[화면 하단 지표 세 개를 왼쪽부터 짚으며] 그 결과가 아래 숫자 세 개입니다. 효과 SO 파생이 **61종**까지 늘었는데, **새 효과를 추가할 때 매니저를 고친 양은 0**줄입니다. 연구 트리 **38노드**를 **전면 개편**할 때도 코드가 아니라 **데이터 작업**만 했습니다.

▶ 슬라이드 18 — 데이터 SO 조합 · 실제로 만들어지는 과정 (1:20) ★

[화면] 슬라이드 18 - 좌: 화염 타워 체인 + 지표 / 우: 팔라딘 조합 + 스프라이트 등식

앞 슬라이드가 "이렇게 설계했다"였다면, 이 장은 실제 에셋 이름과 수치를 그대로 보여주는 장입니다. 글이 거의 없으니 다이어그램을 위에서 아래로 짚으면서 말하세요.

방금 말씀드린 조립 방식이 실제로 어떻게 생겼는지 보여드리겠습니다. 저희 프로젝트에 실제로 들어 있는 에셋이고, 화면의 수치는 인스펙터 값 그대로입니다.

[왼쪽 위에서 아래로 짚으며] 화염 타워 하나를 만드는 데 에셋 다섯 개가 쓰입니다.

TD_FireTower가 타워의 정체입니다. 체력 120, 인구 6이 여기 들어 있습니다. 이게 공격 에셋 TA_FireTower를 참조합니다. 사거리 4에 공격 간격 1초. 그 공격이 다시 효과 두 개를 참조합니다. 하나는 TAE_FireDamage — 즉시 피해 10. 다른 하나는 TAE_FireBurn — 이걸 피해를 주는 게 아니라 상태이상을 붙이는 효과입니다. 그래서 마지막으로 TS_FireBurn을 참조합니다. 3초 동안 1초에 1씩.

[강조] 여기서 코드는 한 줄도 등장하지 않습니다. 전부 인스펙터에서 에셋을 콧아 만든 트리입니다. 얼음 타워는 TAE_IceSlow와 TS_IceSlow로, 돌 타워는 피해 효과만으로 같은 자리에 다른 에셋을 콧은 것입니다. 속성 타워 5종의 구조는 완전히 같고, 조합만 다릅니다.

[오른쪽으로 옮기며] 몬스터도 같은 방식입니다. 예로 팔라딘을 보여드리겠습니다.

근접 공격 MA_CloseRange는 새로 만든 게 아니라 몬스터 16종이 공유하는 에셋입니다. 치유 오라 행동은 힐러에서 쓰던 것을 그대로, 보호 오라 행동은 보호형에서 쓰던 것을 그대로 가져와 배열에 두 개 넣었습니다.

[강조] 오른쪽 아래 그림을 보십시오. 힐러 더하기 보호형이 그대로 팔라딘입니다. 실제 게임에 들어간 스프라이트 그대로 가져온 것이고, 팔라딘을 위해 추가한 코드는 0줄입니다. 기존 두 행동 SO를 배열에 담은 것이 전부입니다.

[오른쪽 아래 "0줄 - 추가한 코드" 지표를 가리키며] 화면에 적힌 이 0줄이 이 장의 결론입니다. 같은 방식으로 쌓인 SO 에셋이 Assets/Data 아래에 769개인데, [강조] 콘텐츠를 늘린 만큼 코드가 늘어나지 않는 구조라는 뜻입니다.

발표 팁 — 이 슬라이드는 앞 슬라이드(컴포지션)의 증거입니다. 앞에서 "행동까지 데이터로 뺐다"고 주장하고, 여기서 실제 에셋 이름으로 보여주는 순서입니다. 시간이 밀리면 왼쪽 타워 체인만 말하고 오른쪽은 "몬스터도 같은 방식입니다" 한 문장으로 넘기세요.

▶ 슬라이드 19 — 이벤트 기반 아키텍처 · 4계층 구조 (1:00)

[화면] 슬라이드 19 - 표현 / 시스템 / 규칙 / 데이터 4단 수직 스택 + 왼쪽 "의존" 화살표

"왜 나뉘는지 → 어떻게 나뉘는지 → 그래서 뭘 얻었는지" 세 덩어리로 말하세요. 계층 이름만 읽으면 남는 게 없습니다. 스택은 맨 아래 데이터부터 위로 짚습니다.

① 왜 나뉘는지

지금까지가 개별 시스템을 어떻게 조립했는지였다면, 이번에는 전반적인 코드의 구조입니다.

[강조] 개발을 하면서 UI에 자원 보유량, 예상 소모량, 타워의 스탯등을 표시하는 계산식이 UI에도, 매니저에도 따로 있는 상태였는데, 저희는 그 문제를 코드를 네 계층으로 나누고, 계산식은 그중 한 곳에만 두고 관리하기로 했습니다.

② 어떻게 나뉘는지

[맨 아래 칸부터 위로 쏘으며] 데이터 계층은 ScriptableObject와 DataTable로 수치를 정의합니다. 코드 수정 없이 교체됩니다.

규칙 계층이 그 핵심입니다. PopulationUpkeepRules(인구 유지비용 규칙)이나 타워유지비용 규칙 같은 **static** 순수 함수 클래스 15개로, 상태도 씬도 갖지 않습니다. 계산식은 전부 이 층에만 있습니다.

시스템 계층은 런타임 동작과 오브젝트 수명만 관리하고, 값은 규칙 계층에 물어봅니다.

표현 계층은 이벤트를 구독만 합니다. 성이 피해를 입으면 이벤트가 나가고 체력바가 그것을 받습니다. 성은 체력바를 모릅니다.

[강조] 왼쪽 "의존" 화살표대로, 윗층은 아래층만 참조하고 아래층은 위층을 아예 모릅니다. 단방향 의존성 설계입니다.

③ 그래서 뭘 얻었는지

[하단 한 줄을 가리키며] 첫째, 같은 계산은 어디서 부르든 같은 답을 냅니다. 인구 유지비는 정산과 다음 날 예측, HUD 표시 세 곳에서 쓰이는데 셋 다 같은 함수를 부릅니다. "표시는 4, 실제로는 8"이 구조적으로 나올 수 없습니다.

둘째, 규칙 계층은 씬 없이 실행됩니다. 게임을 켜지 않고 함수만 호출해 검증할 수 있어 단위 테스트 346개를 확보했고, 밸런스 수식을 고치면 게임을 켜기 전에 테스트가 먼저 잡아냅니다.

발표 팁 — 시간이 밀리면 ②의 네 계층은 이름만 읽고 넘기세요. ①의 "표시는 4, 실제로는 8" 과 ③의 테스트 346개만 남으면 이 장은 성공입니다. 여유가 있으면 표현 계층에서 "이벤트 선언 120곳 이상, 구독 연결 약 400건" 을 한 문장 보태세요.

▶ 슬라이드 20 — 에디터 툴 파이프라인 (0:40)

[화면] 슬라이드 20 - 상단 파이프라인 5단계 + 하단 안내 배너

이 장은 파이프라인 전체 그림만 보여줍니다. 주의 — 하단 배너에는 아직 "다음 두 슬라이드에서 자세히 다뤄니다"라고 적혀 있는데, 22번(도구 실행 화면)이 추가되면서 실제로는 21·22·23번 세 장으로 이어집니다. 배너 문구를 "다음 세 슬라이드"로 고치는 것을 권장하고, 아래 대본은 세 장 기준입니다.

저희는 작업의 속도와 정확도 상승 및 실수를 줄이기 위해서 몇가지 에디터 툴들과 AI스킬을 직접 제작하고 개발 파이프라인에서 적극 활용하였습니다.

그림에 보이는 것처럼 구글시트나 코드등으로 데이터를 정의하고 저희가 제작한 툴들로 에셋화하고 검증까지 마쳐 실제 게임 개발 및 플레이에 적용하였습니다.

저희가 직접 만든 개발 도구를 말씀드리겠습니다.

[강조] 70노드짜리 스킬트리 에셋을 손으로 쓰는 건 저희 개발 기간 내에선 불가능했습니다. 플레이어가 원하는 빌드와 전략을 다양하게 펼쳐주게 하기 위해서 용 스킬트리와 연구 트리에 다양한 선택권이있어야 했고 그래서 저희는 도구를 만들고 콘텐츠를 찍어냈습니다.

[상단 흐름을 가리키며] 파이프라인은 정의 → 자체 도구 → 에셋 → 검증 → 런타임 순서입니다. 가운데 자체 도구와 검증 두 단계가 팀이 직접 만든 구간입니다. 자체 제작한 도구가 22개입니다.

[하단 배너를 가리키며] 이 두 단계를 다음 세 장에서 보겠습니다. 21번은 도구가 무엇을 하는지, 22번은 그 도구가 실제로 어떻게 생겼는지, 그리고 23번이 검증 단계입니다.

▶ 슬라이드 21 — 자체 제작 에디터 도구 (0:40)

[화면] 슬라이드 21 - 에셋 생성 / 레벨 디자인 / 배선 자동화 3카드

앞 장 파이프라인의 '**자체 도구**' 단계를 카드로 펼친 장입니다. 카드를 왼쪽부터 하나씩 짚으세요. 화면 위쪽에 **Unity 메뉴 스크린샷**이 걸려 있습니다. 이 도구들이 실제로 에디터 메뉴에 등록돼 있다는 증거이니, 첫 문장에서 한 번 가리켜 주세요.

[상단 **Unity 메뉴 스크린샷**을 가리키며] 먼저 화면 위쪽을 봐주십시오. 저희 프로젝트의 Unity 메뉴입니다. **여기 달려 있는 항목들이 전부 팀이 직접 만들어 등록한 도구**입니다.

[첫 번째 카드, 에셋 생성을 짚으며] 용 스킬트리와 연구 트리 생성기입니다. 70노드와 38노드 SO를 코드로 생성합니다. [강조] 재실행해도 기존 에셋의 **GUID**를 유지하는 맥등 설계입니다. 그래서 참조가 끊기지 않고 반복해서 다시 구울 수 있습니다.

[두 번째 카드, 레벨 디자인으로 옮기며] 레벨 디자인도 같은 성격입니다. 청크 레이아웃은 브러시로 칠하는 전용 창을 만들었습니다. 코드를 건드리지 않고 맵을 시각적으로 배치합니다.

[세 번째 카드, 배선 자동화로 옮기며] 배선 자동화는 타워 효과음과 건물 프리팹 등록을 일괄로 처리하는 도구입니다. 등록을 빠뜨리는 사고 없이 한 번에 연결합니다.

[하단 배너를 가리키며] 세 도구의 공통점은 **코드나 구글 시트에 적힌 정의를 읽어 SO와 Data Table, 씬 에셋을 자동으로 만들어낸다**는 것입니다. 사람이 하는 일은 정의를 쓰는 것까지고, 그 뒤는 도구가 합니다.

[다음 장으로 넘기며] 말로만 하면 잘 와닿지 않으니, 이 도구들을 **실제로 어떻게 사용했는지** 다음 장에서 보여드리겠습니다.

▶ 슬라이드 22 — 자체 제작 에디터 도구 · 실행 화면 (0:35)

[화면] 슬라이드 22 - 세 도구의 실제 실행 화면 3장

발표 전 반드시 확인 — 세 칸이 아직 **이미지 자리(점선 프레임)** 상태라면 캡처를 넣고 프레임을 지운 뒤 발표하세요. 비어 있는 채로 띄우면 안 됩니다. 이 장은 **설명이 아니라 증거**입니다. 말은 짧게, 화면을 보게 두세요.

앞 장에서 말씀드린 세 도구가 실제로 이렇게 생겼습니다.

[왼쪽 화면을 짚으며] 트리 생성기입니다. 버튼 한 번으로 70노드와 38노드 에셋이 한꺼번에 만들어집니다.

[가운데 화면으로 옮기며] 청크 레이아웃 편집 창입니다. 지형을 **브러시로 칠하듯** 배치합니다.

[오른쪽 화면으로 옮기며] 카탈로그 배선 도구입니다. 효과음과 프리팹 등록을 한 번에 처리합니다.

[강조] 세 화면 모두 상용 에셋이 아니라 팀이 직접 만든 에디터 창입니다. 콘텐츠를 손으로 찍어내는 대신 **찍어내는 기계를 먼저 만들었다**는 것이 이 두 장의 요점입니다.

▶ 슬라이드 23 — Claude (AI) 스킬 제작 (1:00)

[화면] 슬라이드 23 - 스킬 2종 카드 (좌: run-wiring-checker / 우: stringtable-diff) + 각 카드 안 실행 결과 캡처

카드가 두 장으로 정리됐습니다. 각 카드 안에 실행 결과 캡처가 들어가 있으니 설명을 마친 뒤 **캡처를 한 번 가리켜** 주세요. 세 번째 스킬(기여 정리)은 카드가 없으므로 슬라이드를 넘기기 직전에 **구두로 한 문장만** 붙입니다.

앞의 도구들이 **에디터 안에서 도는 것이었다면**, 이번 두 가지는 유니티 에디터 외부에서 생산성을 올려주는 **Claude Code가 실행하는 스킬로** 만들었습니다.

SKILL.md라는 규칙 문서와, 실제로 숫자를 세는 PowerShell 또는 C# 로 작동하게했습니다. 쉽게 말해서 **판단이 필요한 부분은 AI가 하고, 집계는 스크립트가 합니다.**

[첫 번째 카드를 짚으며] **품질 검사 — run-wiring-checker**입니다. Unity 팀 작업에서 가장 흔한 사고가 **인스펙터 연결을 빠뜨리는 것**입니다. 런타임 가드는 그 코드가 실행돼야 알려주지만, 이 스킬은 **플레이하지 않고 빌드 씬을** 정적으로 순회합니다. 미연결 참조, Missing Script, 끊어진 UnityEvent 리스너를 찾아 리포트 파일로 뽑습니다. Coplay MCP로 에디터에 스크립트를 넣어 실행하는 구조입니다. [카드 안 리포트 캡처를 가리키며] 결과는 이렇게 리포트로 나옵니다. 컴파일 에러 0건, 경고는 분류까지 해서 보여줍니다.

[두 번째 카드로 옮기며] **로컬라이제이션 — stringtable-diff**입니다. 저희 언어 원본은 구글 스프레드시트이고, 저장소의 CSV는 사본입니다. 그런데 작업 중에는 로컬 CSV에만 키가 늘어나서, 시트와 어긋나기 쉽습니다. 그래서 **git diff로 새로 생긴 키만** 뽑아 시트에 붙여넣을 TSV로 클립보드에 담습니다. 사람은 시트에서 Ctrl+V만 하면 됩니다. 번역이 아직 안 된 [TBD] 값은 자동으로 표시해서, 미확정 문구가 빌드에 섞이는 걸 막았습니다. [카드 안 실행 결과 캡처를 가리키며] 실행하면 이렇게 **변경된 키가 몇 개인지, 어떤 키인지까지** 알려줍니다.

[슬라이드에는 없으니 구두로 한 문장] **같은 방식으로 만든 스킬이 하나 더 있습니다. 기여 정리 스킬**입니다. git 이력에서 각자 본인의 기여만 집계해 문서로 만드는데, **모든 주장에 커밋 해시나 PR 번호를 근거로 달게** 규칙을 박아뒀습니다. 설명 평가에 들어가는 문서라 근거를 댈 수 없는 문장은 아예 쓰지 않도록 한 것이고, **앞의 11·12·13번 슬라이드에 쓴 기여 분석이 이 스킬의 결과물**입니다.

[마무리하며] 이 스킬들과 **CLAUDE.md** 규칙 문서까지 저장소에 함께 커밋했습니다. **사람과 AI가 같은 규칙 문서를 보고 작업하게 만든 것이** 9주 동안 코드 일관성을 지키는 데 크게 도움이 됐습니다.

발표 팁 — "AI를 썼다"가 아니라 *"AI가 반복할 수 있게 규칙과 스크립트를 만들었다"***가 이 장의 요점**입니다. 시간이 부족하면 **와이어링 체커만 상세히** 말하고, 나머지 둘은 한 문장씩 넘기세요.

▶ 슬라이드 24 — 런타임 성능과 리소스 최적화 (1:00)

[화면] 3축 요약 - 메모리 / 낭비 제거 / 구조

슬라이드는 축마다 한두 줄입니다. **아래 내용을 구두로** 채우세요.

성능은 나중에 프로파일러로 잡는 것보다, 처음부터 **문제가 생기지 않게 설계하는 게** 낫다고 봤습니다. 서로 다른 세 가지 문제를 각각 다뤘습니다.

[강조] 첫째, **메모리**입니다. 생성과 파괴 자체를 없앴습니다. Unity에서 **Instantiate**와 **Destroy**는 비싸고, 특히 파괴는 가비지를 남겨 나중에 프레임이 끊깁니다. 그래서 오브젝트 풀링을 썼는데, **하나로 뭉개지 않고 수명 특성에 따라 두 종류로 나눴습니다.** **ComponentPool**은 인덱스형입니다. 점령 테두리처럼 매번 전체를 다시 그리는 렌더러용이라, 앞의 N개만 쓰고 뒤는 끕니다. **PrefabPool**은 획득·반납형입니다. 주민과 병사, 그리고 투사체는 각자 태어나고 사라지는 시점이 다르니까요. 겉모습이 열 종류라 프리팹별로 통을 나눠서, 병사를 요청했는데 주민이 나오는 일이 없게 했습니다.

[강조] 둘째, **피할 수 없는 작업의 낭비를 줄였습니다.** 몬스터는 계속 움직이니까 정렬 계산을 매 프레임 **할 수밖에 없습니다.** 대신 계산 결과가 바뀌었을 때만 **sortingOrder**에 값을 씁니다 — 렌더러 속성을 건드리는 건 비싸

거든요. 그리고 안 움직이는 건물은 아예 매 프레임 검사에서 빼고, 배치할 때 한 번만 정렬합니다. 비동기 처리도 Coroutine을 전면 폐기하고 UniTask로 바꿔 대기 객체 할당을 없앴습니다. **StartCoroutine** 사용 **0건**입니다.

셋째는 구조입니다. **Update()**를 쓰는 파일이 675개 중 58개, 8.6%입니다.

[강조] 여기서 솔직하게 말씀드릴 부분이 있습니다. 이 비율에는 저희 게임의 낮 운영이 **이산 이벤트로 표현되는 장르 특성**도 함께 작용했습니다. 액션 게임이었으면 이 숫자는 안 나옵니다. 그래서 이건 성과라기보다 **지표**에 가깝습니다.

다만 같은 장르에서도 풀링으로 짜는 경우가 많습니다. 저희는 "**이벤트 구독자는 구독 직후 현재 값을 한 번 반영한다**"는 규칙까지 문서로 정했습니다. 풀링을 쓰면 애초에 필요 없는 고민인데, **초기값 동기화 문제를 감수하고서라도 이벤트로 가겠다**고 판단한 겁니다. 그 규칙을 675개 파일 내내 지켰습니다.

발표 팁 ① — 셋째 축의 "장르 특성도 작용했다"를 **먼저 인정하고 들어가세요**. 질문받고 인정하면 변명이 되지 만, 먼저 말하면 자기 인식으로 들립니다. **이 문장은 슬라이드에 없습니다**. 화면에는 **Update() 사용 58개 / 전체 675개 파일** 한 줄뿐이니 그 숫자를 짚은 직후에 **구두로** 붙이세요.

발표 팁 ② — 시간이 부족하면 **메모리 축만** 자세히 말하고 나머지 둘은 한 문장씩 넘기세요. 풀링을 두 종류로 나눈 판단이 이 슬라이드에서 가장 강한 내용입니다.

발표 팁 ③ — 시간이 되면 **Unity Profiler로 실제 수치를 재서 넣으시길 권합니다**. 밤 웨이브 최대 부하 구간에서 풀링 적용 전후의 GC Alloc 차이 한 줄이면 이 슬라이드가 완전히 달라집니다.

▶ 슬라이드 25 — 성능 트러블슈팅 사례 (1:20) ★

[화면] 슬라이드 25 - 증상 → 해결 → 결과 세로 흐름 + 오른쪽에 원인 박스

화살표를 따라 **증상 → (오른쪽 원인) → 해결 → 결과** 순서로 짚으세요. **162ms는 화면 하단에 큰 지표로 이미 올라가 있습니다**. 마지막에 **그 숫자를 직접 짚으면서** 해석을 붙이세요. 이 장의 유일한 실측치입니다.

[증상을 짚으며] 앞 장까지가 "이렇게 설계했다"였다면, 이 장은 **실제로 터진 문제를 잡은 사례**입니다. 증상은 단순했습니다. **오라 타워를 맵에 놓을수록 프레임이 급격히 떨어졌습니다**.

[원인을 짚으며] 저희 영혼 타워는 주변 타워에 **인구를 얹어 주는 오라**를 뱉습니다. 그런데 타워가 가동 중인지 판정하려면 **오라로 받은 인구까지** 세야 하고, 오라를 뱉는 쪽도 **자기가 가동 중이어야** 오라가 나갑니다.

[강조] "**가동 중인가**"와 "**오라를 받았는가**"가 서로를 물고 도는 **순환**이 생긴 겁니다.

처음에는 재진입 가드로 막아 크래시는 없었습니다. 하지만 이 가드는 **같은 타워로 되돌아오는 것만** 끊고, 다른 타워로 내려가는 분기는 끊지 못합니다. 그래서 한 번의 조회가 오라 타워 개수의 **순열만큼** 번졌습니다. 팩토리얼입니다. **타워가 하나 늘 때마다 비용이 폭발한** 겁니다.

[해결을 짚으며] 그래서 재귀를 줄인 게 아니라 **아예 없앴습니다**. 프레임당 한 번, 어느 타워가 어떤 보정을 받는지를 **목록으로 한꺼번에 만들어 두고**, 그 뒤의 모든 조회는 **그 목록에서 꺼내 쓰기만** 합니다. 계산을 다시 하지 않습니다.

[강조] 핵심은 **계산을 시작하기 전에 프레임 번호를 먼저 찍는다는 점**입니다. 목록을 채우는 도중에 누가 되짚어 들어와도 **목록을 읽고 즉시 돌아갑니다**. 가드로 막은 게 아니라, **재귀가 아예 불가능한 구조로 바꾼** 겁니다.

목록은 처음에 전부 "**보정 없음**"으로 깔아 둡니다. 오라 효과는 **검찰수록 커지기만 하고 줄지 않기 때문에**, 거기서 시작한 계산은 한 방향으로만 올라가다 멈춥니다. **어떤 순서로 계산하든 답이 같습니다**.

[결과 줄을 짚으며] 덤으로 정확해지기까지 했습니다. 예전에는 영혼 타워끼리 겹쳐 놓으면 계산 순서에 따라 값이 달라지는 근사였는데, 이제는 하나로 확정됩니다.

[화면 하단의 162ms 지표를 직접 짚으며] [강조] 마지막으로 화면 아래 수치 하나만 짚고 넘어가겠습니다. 고치기 전, 오라 타워 5개에 타워 10개를 놓은 배치에서 프레임당 162밀리초가 나왔습니다. 화면에 함께 적혀 있듯 60fps 기준 한 프레임 예산이 16.7밀리초니까, 약 열 배를 넘긴 상태였습니다.

질문이 나오거나 여유가 있으면 — 기술 부채 이야기

재진입 가드를 넣을 당시, 코드 주석에 "문제가 확인되면 고정점 반복으로 다시 봐야 한다" 고 적어뒀고, 사흘 뒤 실제로 문제가 드러나자 그대로 실행했습니다. (08-22 → 08-25) 부채를 기록해 두었다가 계획대로 해소한 사례이고, 커밋으로 근거를 댈 수 있습니다.

예상 질문 대비 한 줄씩 · 목록이 정확히 뭔가? — "타워 → 그 타워가 받는 공격력·공격속도·보호막·추가 인구" 를 담은 딕셔너리입니다. · 캐싱으로는 안 됐나? — 순환 자체가 남아 캐시 히트 순서에 따라 답이 달라집니다. · 매 프레임 전체 계산이 더 무겁지 않나? — 최악이 팩토리얼에서 다항으로 내려왔고, 이후 조회는 표 읽기 한 번입니다. · 수렴 안 하면? — 경고 로그를 남기고 마지막 값으로 확정합니다. 멈추거나 튀지 않습니다.

발표 팁 — 이 장이 04장에서 가장 강합니다. 유일한 실측 수치(162ms) 가 있고, 미시 최적화가 아니라 알고리즘 복잡도를 바꾼 사례이기 때문입니다. 시간이 밀려도 지키시고, 밀리면 "멈춘다는 보장" 문단만 생략하세요.

▶ 슬라이드 26 — 적용한 디자인 패턴 (1:30)

[화면] 슬라이드 26 — 패턴 8종 2열 표 (패턴 / 해결한 문제)

슬라이드는 8행 표입니다. 여덟 개를 다 읽으면 시간이 넘어갑니다. 오른쪽 "해결한 문제" 열만 훑고, Null Object 설명은 구두로 붙이세요.

[표의 오른쪽 열을 위에서 아래로 훑으며] [강조] 저희는 패턴을 먼저 정해두고 끼워 맞추는 게 아니라, 실제로 겪은 문제마다 맞는 패턴을 골랐습니다. 부제에 적힌 대로, 패턴을 쓴 것이 아니라 겪은 문제를 패턴으로 푼 것입니다.

효과 종류마다 매니저에 switch 분기가 쌓이던 문제는 Strategy로 풀었습니다. 효과 SO 파생이 61종입니다. 한 스탯에 네 시스템이 동시에 개입하는 문제는 Composite로, 여러 보정치를 질의해 합성합니다. 용 트리와 연구 트리가 순회 로직을 중복 구현하던 것은 Template Method로 골격만 공유하게 했습니다. 연출 오브젝트의 반복 생성·파괴 비용은 Object Pool로, 시스템 간 직접 참조는 Observer로 끊었습니다. 낮/밤 전이 규칙이 여러 곳에 흩어진 문제는 State로 일원화했고, 런타임 객체와 저장 포맷의 결합은 DTO로 분리해 세이브를 안정화했습니다.

[강조] 네 번째 줄 Null Object — 여기만 조금 자세히 말씀드리겠습니다. 특수 타워는 인구 없이 돌아가야 합니다. 이걸 조건 분기로 처리하면 타워 코드 곳곳에 예외가 생깁니다. 그래서 ITowerStaffing을 구현하되 항상 100퍼센트를 반환하는 AlwaysOnStaffing을 만들어 붙였습니다. 같은 방식으로, 점령 강화 대상에서 제외해야 하는 몬스터는 중립 스냅샷을 돌려줍니다. 결과적으로 Tower 클래스에는 예외 분기가 하나도 없습니다. 예외를 분기로 처리하지 않고 타입으로 처리한다 — 이게 저희가 이 프로젝트에서 배운 것 중 하나입니다.

[여유가 있으면 보태는 한 줄] Object Pool은 연출과 투사체의 GC 할당을 없앴 것이고, Observer는 이벤트 기반으로 시스템 간 결합을 끊은 것입니다. 앞의 24번 슬라이드에서 이미 나온 이야기입니다.

04 · 완성도 · 시연 (1:20)

▶ 슬라이드 27 — 완성도 보강 (0:50)

[화면] 슬라이드 27 - 온보딩 / 밸런싱 / 안정화 아이콘 3열 + 요약 카드

슬라이드는 축마다 **한 줄 목표 + 항목 요약**입니다. 카드의 항목을 읽지 말고, 아이콘을 짚으며 **아래 내용을 구두**로 채우세요.

알파 이후에는 새 기능을 더하는 대신 **완성도**에 집중했습니다. 세 방향입니다.

[첫 번째 아이콘을 짚으며] **첫째, 온보딩입니다.** 저희 게임은 규칙이 많아서 아무 안내가 없으면 처음 하는 사람이 무엇을 해야 할지 모릅니다. 그래서 본게임과 분리된 **튜토리얼 전용 씬**을 만들어 1~3일차를 12개 챕터로 나눴습니다. 필수 목표를 끝내지 않으면 밤 진입을 막고 남은 목표 수를 안내합니다. 튜토리얼을 건너뛴 사람도 본게임 안에서 안내 받도록 **가이드 퀘스트 28종**을 넣었고, 처음 마주친 시스템을 자동으로 발견 처리해 팝업을 띄우는 **도움말 도감 16종**을 준비했습니다.

[두 번째 아이콘으로 옮기며] **둘째, 밸런싱입니다.** 알파 피드백을 반영해 초반 난이도를 완화했습니다. 기본 타워 세 종의 공격 간격과 건설 비용을 낮추고, 시작 자원과 인구를 조정하고, 성 근처 점령지는 점령 기간을 하루 단축했습니다. 점령 패널티는 스탯 배율에서 추가 스폰 중심으로 바꿨고, 최종 보스 데이터도 보정했습니다. [강조] 이 과정에서 **밸런스 시뮬레이터**를 직접 만들어 수치를 검증했습니다. Unity 에셋에서 수치를 굵어 단일 HTML로 굽고, 회귀 테스트 다섯 종을 붙여 조정이 상식적인 방향으로 반응하는지 확인했습니다.

[세 번째 아이콘으로 옮기며] **셋째, 안정화입니다.** 낮 정산 경계를 기준으로 삼아 세이브·로드 파이프라인을 구축했습니다. 자동저장과 수동저장을 지원하고, 건물 위치와 회전, 인구, 비활성 타워 상태까지 복원합니다. 설정창과 키 리바인딩, 사운드 매니저도 이 시기에 통합했고, 주민 프리팹에 오브젝트 풀링을 적용했습니다.

▶ 슬라이드 28 — 시연 영상 (0:30)

[화면] 슬라이드 28 - DEMO

시연 영상은 별도 파일로 제출했습니다.

낮 운영에서 건설과 인구 배치를 보여드리고, 밤 방어에서 웨이브 대응과 타워 비활성화, 점령을 통한 영토 확장, 어미용 속성 변경과 궁극기, 보스 웨이브, 그리고 승리 조건 달성까지의 순서로 구성했습니다.

▶ 슬라이드 29 — 시연 영상 전체 화면

[분기] 영상을 이 자리에서 재생하는 경우 - "지금 함께 보시겠습니다." 하고 재생

05 · 자체 평가 의견 (2:05)

▶ 슬라이드 30 — 완성도 평가 (0:50)

[화면] 슬라이드 30 - 9.5/10 + 기획 의도 부합도 · 확장 가능성 · 10점을 주지 못한 이유

저희 팀이 스스로 매긴 완성도는 **10점 만점에 9.5점**입니다.

그렇게 준 이유는, 낮과 밤 루프부터 인구 배분, 타워와 전투, 점령, 용, 연구, 세이브·로드까지 핵심 시스템이 전부 연결되어 있고, 28일 1회차 완주와 두 가지 승리 경로 도달이 실제로 가능한 상태이기 때문입니다.

기획 의도 부합도 측면에서는, "낮의 선택이 밤의 생존을 결정한다"는 컨셉이 말로만 남지 않고 인구 배분, 타워 충원율, 점령 페널티라는 세 개의 연동 시스템으로 실제 구현됐다고 봅니다.

확장 가능성 측면에서는, 모든 밸런스 수치가 데이터 에셋으로 분리돼 있어 **코드 수정 없이 콘텐츠 추가와 수치 조정, 신규 속성 확장이 가능한 구조**라는 점을 들 수 있습니다.

[화면 아래 "10점을 주지 못한 이유"를 가리키며] **만점을 주지 못한 이유**는 화면에 적은 그대로입니다. 시스템은 전부 붙었지만, **속성별 용 빌드 트리마다 선택할 만한 메리트가 고르지 않습니다**. 완성된 하나의 "게임"으로서의 밸런스가 아직 아쉽다고 봤고, 그래서 0.5점을 뺐습니다. 남은 마감 과제까지 포함해 다음 슬라이드에서 이어서 말씀드리겠습니다.

▶ 슬라이드 31 — 추후 개선점 및 느낀 점 (0:50)

[화면] 슬라이드 31 - 좌: 개선점 5항목 / 우: 팀원별 느낀 점

발표 전 반드시 확인 — 오른쪽 "느낀 점" 네 칸 중 **세 칸이 아직 [작성 필요]**이고, 나머지 한 칸도 임시 문구입니다. **화면에 [작성 필요]가 보이는 채로 띄우면 안 됩니다**. 네 명의 문장을 채운 뒤 아래 초안을 실제 문구에 맞게 고쳐 읽으세요.

[왼쪽을 짚으며] 남은 과제는 다섯 가지입니다.

첫째, 잔여 공백 마감입니다. 연구 노드 효과 연결, 대공 타워 세이브 ID 등록, 봉인석 해금 연결이 남아 있습니다. **둘째, 완주 QA 확대**입니다. 튜토리얼 1~3일차와 본게임 28일을 반복 검증해야 합니다. **셋째, 성능 최적화 범위 확대**입니다. 오브젝트 풀링을 주민에는 적용했지만 몬스터는 아직입니다. **넷째, 사전 검증 강화**입니다. 구현에 들어가기 전에 소형 프로토타입으로 핵심 재미를 먼저 확인했어야 했습니다. **다섯째, 설계 단계 보완**입니다. 시스템 간 의존 관계를 먼저 확정했다면 재작업과 통합 비용을 크게 줄일 수 있었을 겁니다.

[오른쪽으로 옮기며] 팀원별로 이번 프로젝트에서 얻은 것을 정리했습니다.

[초안 - 실제 작성된 문구로 대체] **조강현 팀장**은 4계층 아키텍처와 톨 파이프라인을 통합 설계하는 역량을 체득했습니다. **김지해 팀원**은 UX 중심의 반응형 UI와 주민 애니메이션 상태 머신 노하우를 확보했습니다. **이하늘 팀원**은 아이소메트릭 그리드와 온보딩 시스템을 완성했습니다. **나상욱 팀원**은 몬스터 34종과 70노드 스킬트리를 데이터 주도로 밸런싱하는 경험을 쌓았습니다.

▶ 슬라이드 32 — Q & A (0:25)

[화면] 마지막 슬라이드 - Q & A + "낮의 선택이 밤의 생존을 결정한다"

저희 3팀의 **Tower and Dragons**는 단순히 타워를 많이 세우는 게임이 아니라, **낮의 선택이 밤의 생존으로 이어지는** 전략 게임을 목표로 만들었습니다.

핵심 키워드는 세 가지입니다. **용의 운용, 인구의 배분, 전략적 확장**. 이 세 요소가 서로 물려 돌아가며 플레이어에게 매일 새로운 전략적 선택을 제공하는 것이 저희 프로젝트의 목표였습니다.

9주 동안 기획한 것을 전부 구현하지는 못했지만, **왜 그렇게 만들었고 왜 그렇게 바꿨는지는 모든 항목에 대해 설명드릴 수 있습니다.**

이상으로 3팀 발표를 마치겠습니다. 감사합니다.

[화면 하단 컨셉 문장을 가리키며] 질문 받겠습니다.